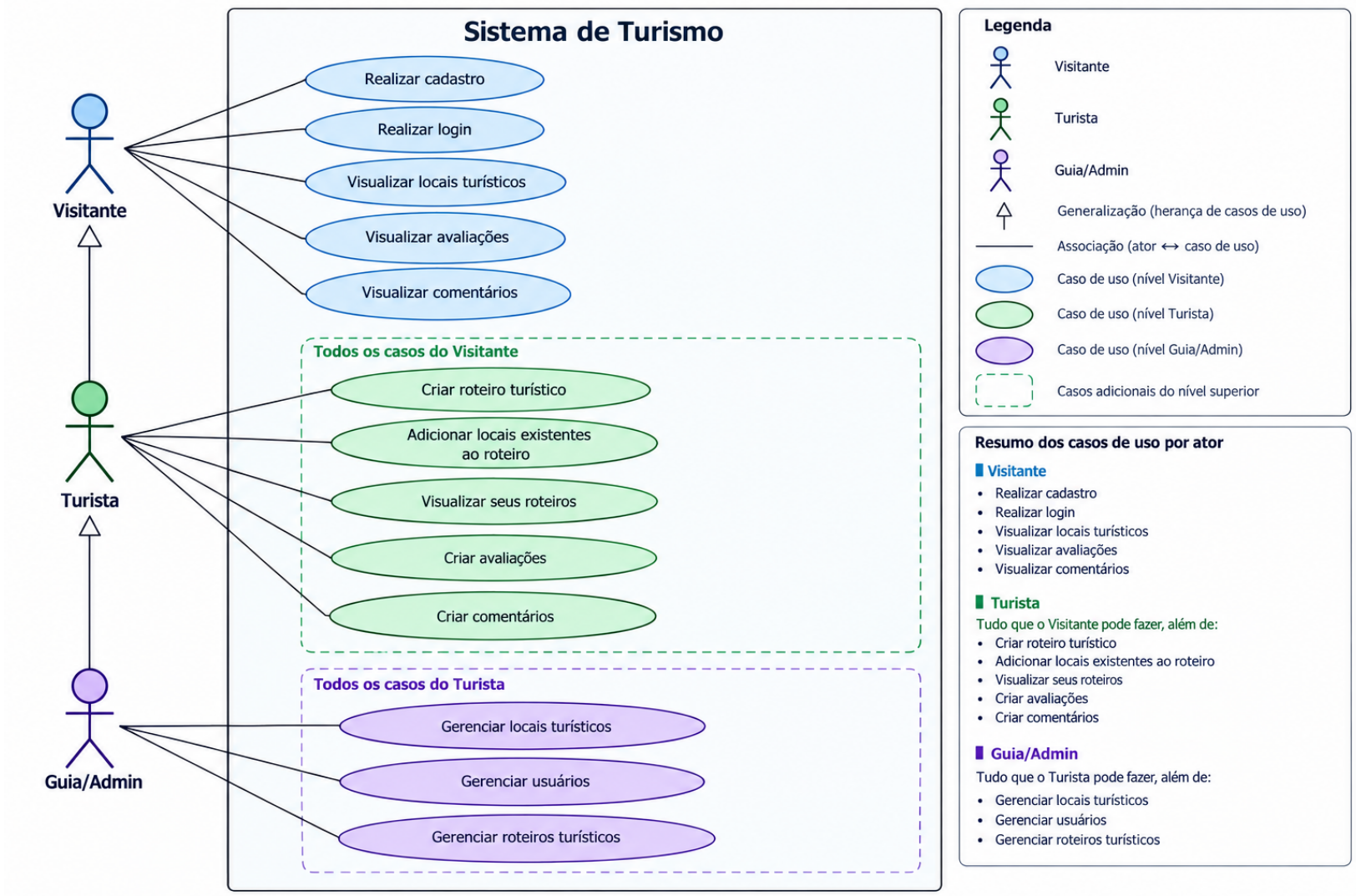
Diagrama de casos de uso

Diagrama original preservado. As próximas páginas detalham os 13 casos.



Descrições dos casos de uso

Mesmos nomes e numeração do diagrama original.

CU-01 - Realizar cadastro

Ator: Visitante

Objetivo: Criar uma conta para utilizar os recursos do sistema.

Pré-condição: Não estar autenticado e informar nome, e-mail e senha na opção de cadastro.

Fluxo principal:

1. Informar nome, e-mail e senha; o código de guia é opcional.
2. O sistema valida os dados e verifica se nome ou e-mail já estão em uso.
3. A senha é protegida e a conta é salva como turista ou guia, conforme o código informado.
4. Após o cadastro, o sistema tenta realizar o login automaticamente.

Alternativas: Dados inválidos ou nome/e-mail já cadastrado impedem o cadastro. Sem código de guia válido, a conta recebe o perfil de turista.

Resultado: Conta criada e disponível para acesso ao sistema.

CU-02 - Realizar login

Ator: Visitante com conta cadastrada

Objetivo: Entrar no sistema e acessar as funções do seu perfil.

Pré-condição: Possuir uma conta e informar e-mail e senha.

Fluxo principal:

1. Acessar a tela de entrada e informar as credenciais.
2. O sistema localiza a conta e confere a senha.
3. Se os dados estiverem corretos, o sistema inicia a sessão.
4. O usuário passa a acessar as funcionalidades permitidas para seu perfil.

Alternativas: E-mail inexistente ou senha incorreta impedem o acesso. Se a sessão expirar, pode ser necessário entrar novamente.

Resultado: Usuário autenticado como turista ou guia/administrador.

Descrições dos casos de uso

Mesmos nomes e numeração do diagrama original.

CU-03 - Visualizar locais turísticos

Ator: Visitante, Turista ou Guia/Administrador

Objetivo: Conhecer os pontos turísticos cadastrados.

Pré-condição: Acessar a página inicial do sistema.

Fluxo principal:

1. O sistema carrega os locais cadastrados e suas coordenadas.
2. Os locais são apresentados em cartões e marcadores no mapa.
3. O usuário escolhe um cartão ou marcador.
4. O sistema exibe nome, descrição, cidade, categoria, localização e imagem, quando houver.

Alternativas: Se não existirem locais, o sistema informa a ausência de registros. Um local removido não pode ter seus detalhes consultados.

Resultado: Informações do local apresentadas ao usuário.

CU-04 - Visualizar avaliações

Ator: Visitante, Turista ou Guia/Administrador

Objetivo: Consultar as experiências e notas de outras pessoas.

Pré-condição: Selecionar um ponto turístico.

Fluxo principal:

1. Abrir a página de detalhes do local.
2. O sistema busca as avaliações vinculadas a esse ponto turístico.
3. As avaliações são exibidas da mais recente para a mais antiga.
4. O usuário visualiza nota, comentário, autor e imagem, quando houver.

Alternativas: Se o local ainda não tiver avaliações, o sistema apresenta uma mensagem informativa.

Resultado: Avaliações consultadas sem alterar os registros.

Descrições dos casos de uso

Mesmos nomes e numeração do diagrama original.

CU-05 - Visualizar comentários

Ator: Visitante, Turista ou Guia/Administrador

Objetivo: Ler o relato escrito por quem avaliou um local.

Pré-condição: Abrir a seção de avaliações de um ponto turístico.

Fluxo principal:

1. O usuário acessa os detalhes do local.
2. O sistema carrega as avaliações e os comentários associados.
3. O texto de cada comentário é apresentado junto à nota e ao autor.

Alternativas: Sem avaliações registradas, não há comentários para apresentar.

Resultado: Comentário lido pelo usuário.

Observação: No projeto, o comentário faz parte da avaliação. Não existe uma publicação separada apenas para comentários.

CU-06 - Criar roteiro turístico

Ator: Turista ou Guia/Administrador

Objetivo: Organizar locais turísticos em um roteiro de viagem.

Pré-condição: Estar autenticado e ter pontos turísticos disponíveis.

Fluxo principal:

1. Escolher a opção de criar roteiro.
2. Informar título, descrição e duração em dias.
3. Selecionar pelo menos um dos locais cadastrados.
4. O sistema valida os dados e salva o roteiro como rascunho, vinculado ao autor.

Alternativas: A tela impede o envio sem locais selecionados. Campos obrigatórios ausentes ou duração inválida impedem a criação.

Resultado: Roteiro criado como rascunho, ainda sem publicação pública.

Descrições dos casos de uso

Mesmos nomes e numeração do diagrama original.

CU-07 - Adicionar locais existentes ao roteiro

Ator: Turista autor do roteiro ou Guia/Administrador

Objetivo: Definir os pontos turísticos que compõem o roteiro.

Pré-condição: Estar autenticado e selecionar locais já cadastrados.

Fluxo principal:

1. O sistema apresenta os locais disponíveis no formulário de criação.
2. O usuário marca os pontos que deseja visitar.
3. A seleção é enviada junto aos dados do roteiro.
4. O sistema associa os locais escolhidos ao roteiro.

Alternativas: Se um local deixar de existir antes de salvar, a associação pode falhar. Um roteiro em análise não pode ser alterado.

Resultado: Locais existentes vinculados ao roteiro.

Observação: A seleção está disponível na criação. A alteração posterior dos locais existe na API, mas ainda não possui uma tela própria.

CU-08 - Visualizar seus roteiros

Ator: Turista ou Guia/Administrador

Objetivo: Consultar os roteiros criados e acompanhar sua situação.

Pré-condição: Estar autenticado no sistema.

Fluxo principal:

1. Acessar a página de roteiros.
2. Para o turista, o sistema lista os roteiros de sua autoria.
3. A lista apresenta título, duração, locais e situação do roteiro.
4. Ao abrir um roteiro, o usuário consulta os detalhes e o motivo de rejeição, se houver.

Alternativas: Se não houver roteiros, o sistema informa a ausência de registros. O turista não pode consultar roteiros privados de outro autor.

Resultado: Roteiros e suas situações apresentados ao usuário.

Observação: A tela do guia apresenta todos os roteiros para gestão e revisão. A consulta exclusiva dos roteiros de sua autoria também está disponível pela API.

Descrições dos casos de uso

Mesmos nomes e numeração do diagrama original.

CU-09 - Criar avaliações

Ator: Turista ou Guia/Administrador

Objetivo: Registrar sua experiência em um ponto turístico.

Pré-condição: Estar autenticado e selecionar um local existente.

Fluxo principal:

1. Abrir os detalhes do local e preencher o formulário de avaliação.
2. Escolher uma nota de 1 a 5 e escrever um comentário.
3. Anexar uma imagem, se desejar, e enviar a avaliação.
4. O sistema valida os dados, armazena a imagem quando enviada e salva a avaliação com o autor.

Alternativas: Nota inválida, comentário vazio ou imagem não permitida impedem o registro. A imagem pode ser JPEG, PNG ou WEBP, com até 5 MB.

Resultado: Avaliação registrada e disponível para consulta pública.

CU-10 - Criar comentários

Ator: Turista ou Guia/Administrador

Objetivo: Descrever a experiência vivida no local avaliado.

Pré-condição: Estar autenticado e preencher uma avaliação do local.

Fluxo principal:

1. Escrever o relato no campo de comentário da avaliação.
2. Informar também a nota do local.
3. Enviar o formulário de avaliação.
4. O sistema salva o comentário junto à nota e à identificação do autor.

Alternativas: A interface não permite comentário vazio ou composto somente por espaços. Sem nota válida, o registro também não é salvo.

Resultado: Comentário publicado como parte de uma avaliação.

Observação: Este caso e "Criar avaliações" são executados no mesmo formulário.

Descrições dos casos de uso

Mesmos nomes e numeração do diagrama original.

CU-11 - Gerenciar locais turísticos

Ator: Guia/Administrador

Objetivo: Manter o cadastro dos pontos turísticos.

Pré-condição: Estar autenticado com o perfil de guia.

Fluxo principal:

1. Escolher a opção de cadastrar ou abrir um local existente.
2. Para cadastrar, informar nome, descrição, cidade e categoria e marcar a posição no mapa; a imagem é opcional.
3. Para editar, alterar os dados e salvar; para excluir, confirmar a operação.
4. O sistema verifica a permissão, valida os dados e realiza a operação.

Alternativas: Turistas não podem executar essas ações. Dados ou imagens inválidos impedem a gravação; a exclusão pode falhar se existirem vínculos no banco.

Resultado: Local cadastrado, atualizado ou excluído.

CU-12 - Gerenciar usuários

Ator: Guia/Administrador

Objetivo: Consultar as contas cadastradas no sistema.

Pré-condição: Estar autenticado com o perfil de guia.

Fluxo principal:

1. Acessar a página de usuários cadastrados.
2. O sistema verifica a permissão do guia.
3. A lista apresenta nome, e-mail, perfil e data de cadastro.
4. O guia consulta as informações necessárias.

Alternativas: Usuários sem o perfil de guia não podem acessar a listagem. Se ocorrer uma falha, o sistema informa o erro.

Resultado: Usuários consultados, sem alteração dos cadastros.

Observação: Na versão atual, este gerenciamento permite somente listar usuários. Editar ou excluir contas ainda não está disponível.

Descrições dos casos de uso

Mesmos nomes e numeração do diagrama original.

CU-13 - Gerenciar roteiros turísticos

Ator: Guia/Administrador

Objetivo: Acompanhar os roteiros e controlar sua publicação.

Pré-condição: Estar autenticado com o perfil de guia.

Fluxo principal:

1. Acessar a página de roteiros e consultar os registros de todos os autores.
2. Abrir um roteiro para verificar descrição, locais, duração, autor e situação.
3. Aguardar o envio para análise pelo próprio autor: um roteiro novo começa como rascunho e só o autor pode solicitar sua revisão.
4. Se estiver em análise, decidir entre publicar e rejeitar. Na rejeição, o guia pode informar um motivo.
5. O sistema salva a decisão. Somente os roteiros publicados ficam disponíveis para consulta pública.
6. Se necessário, despublicar um roteiro publicado. Ele volta a ser rascunho e sai da consulta pública.

Alternativas: Somente roteiros em análise podem ser publicados ou rejeitados. A edição fica bloqueada enquanto o roteiro está em análise, tanto para o autor quanto para o guia. Um roteiro rejeitado pode ser corrigido e reenviado pelo próprio autor. Cancelar uma confirmação ou o diálogo de rejeição não altera o roteiro.

Resultado: Situação do roteiro atualizada conforme a decisão do guia, preservando seu autor e os locais associados.

Observação: Editar e excluir roteiros também são operações disponíveis pela API para o guia, mas ainda não têm controles próprios nessa tela. O guia pode editar roteiros de qualquer autor; o turista pode editar os seus, respeitando o bloqueio durante a análise.

Fluxo de publicação

O autor cria um rascunho e o envia para análise. O guia publica ou rejeita. O autor pode ajustar um rejeitado e reenviar. O guia pode despublicar um publicado, que volta a rascunho.

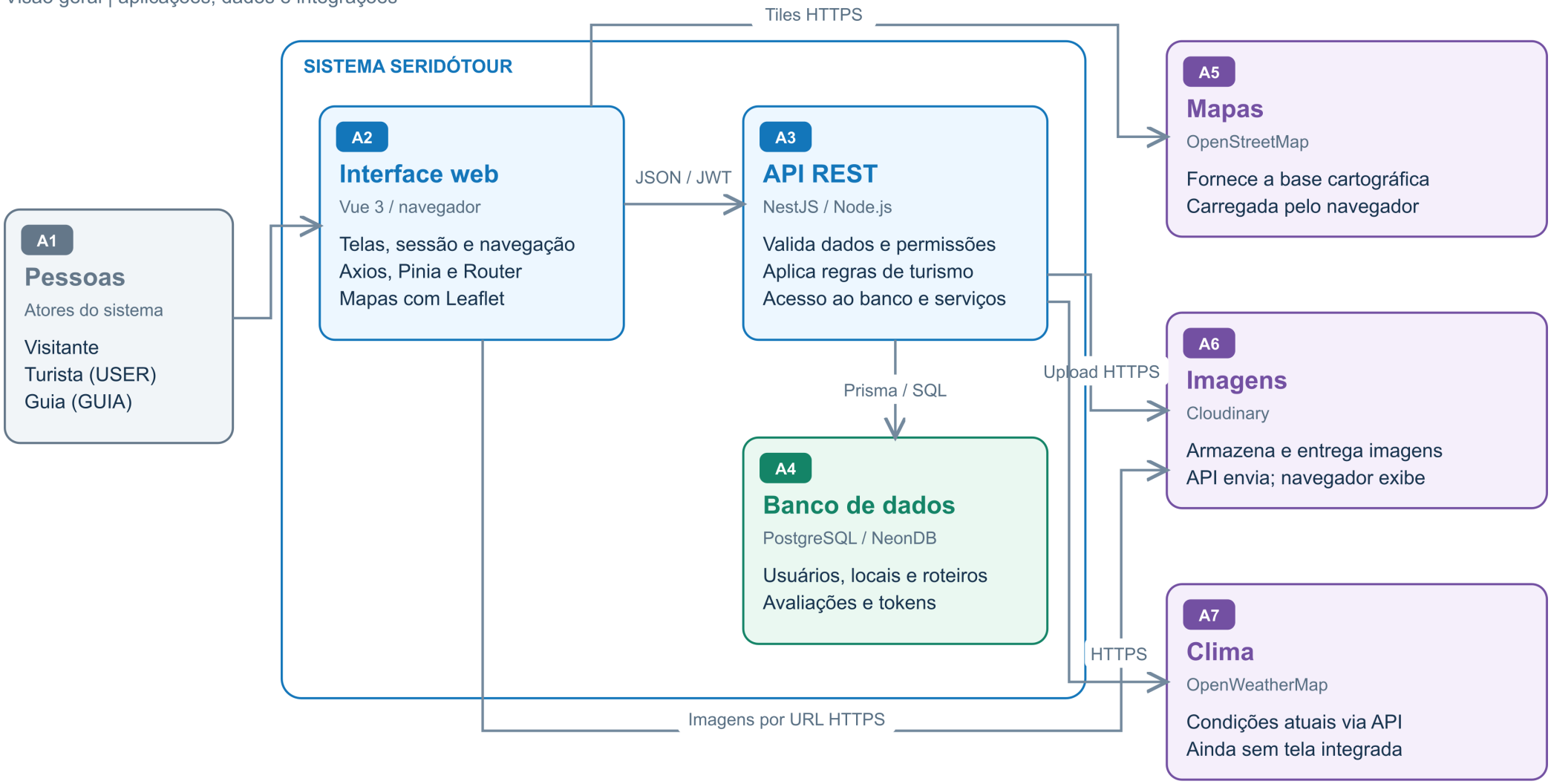
Atores e acesso

Visitante acessa conteúdo público. Turista é o papel USER. Guia/Administrador corresponde ao papel GUIA e possui as funções administrativas. Cadastro e login são destinados a quem ainda não iniciou uma sessão.

SERIDÓTOUR / PROJETO INTEGRADOR

Arquitetura do sistema

Visão geral | aplicações, dados e integrações



Azul: aplicação • Verde: dados • Roxo: serviço externo • Setas: chamadas; respostas retornam ao chamador.

O frontend executa no navegador. O Prisma pertence à API. O navegador não acessa o PostgreSQL.

Arquitetura do sistema

Aplicação cliente-servidor: interface, API, dados e serviços externos.

A1 - Pessoas

Visitante consulta informações públicas; turista cria roteiros e avaliações; guia também gerencia locais, consulta usuários e modera roteiros. No banco, os papéis são USER e GUIA.

A2 - Interface web

Vue 3 apresenta telas e formulários. Vue Router controla a navegação, Pinia mantém a sessão e Axios envia chamadas à API. Leaflet apresenta os mapas. O código da interface executa no navegador.

A3 - API REST

NestJS recebe as chamadas, valida dados e permissões e aplica regras de negócio. JWT identifica o usuário; o papel GUIA libera as operações administrativas.

A4 - Banco de dados

PostgreSQL, hospedado no NeonDB em produção, armazena usuários, tokens, locais, roteiros e avaliações. Prisma faz o acesso ao banco dentro da API. Imagens são armazenadas como URLs.

A5 - OpenStreetMap

Fornece as imagens da base do mapa diretamente ao navegador. Os marcadores usam coordenadas dos locais cadastrados. Não há cálculo automático de trajetos.

A6 - Cloudinary

Recebe imagens enviadas pelo backend. O banco guarda a URL retornada e o navegador usa essa URL para exibir a imagem.

A7 - OpenWeatherMap

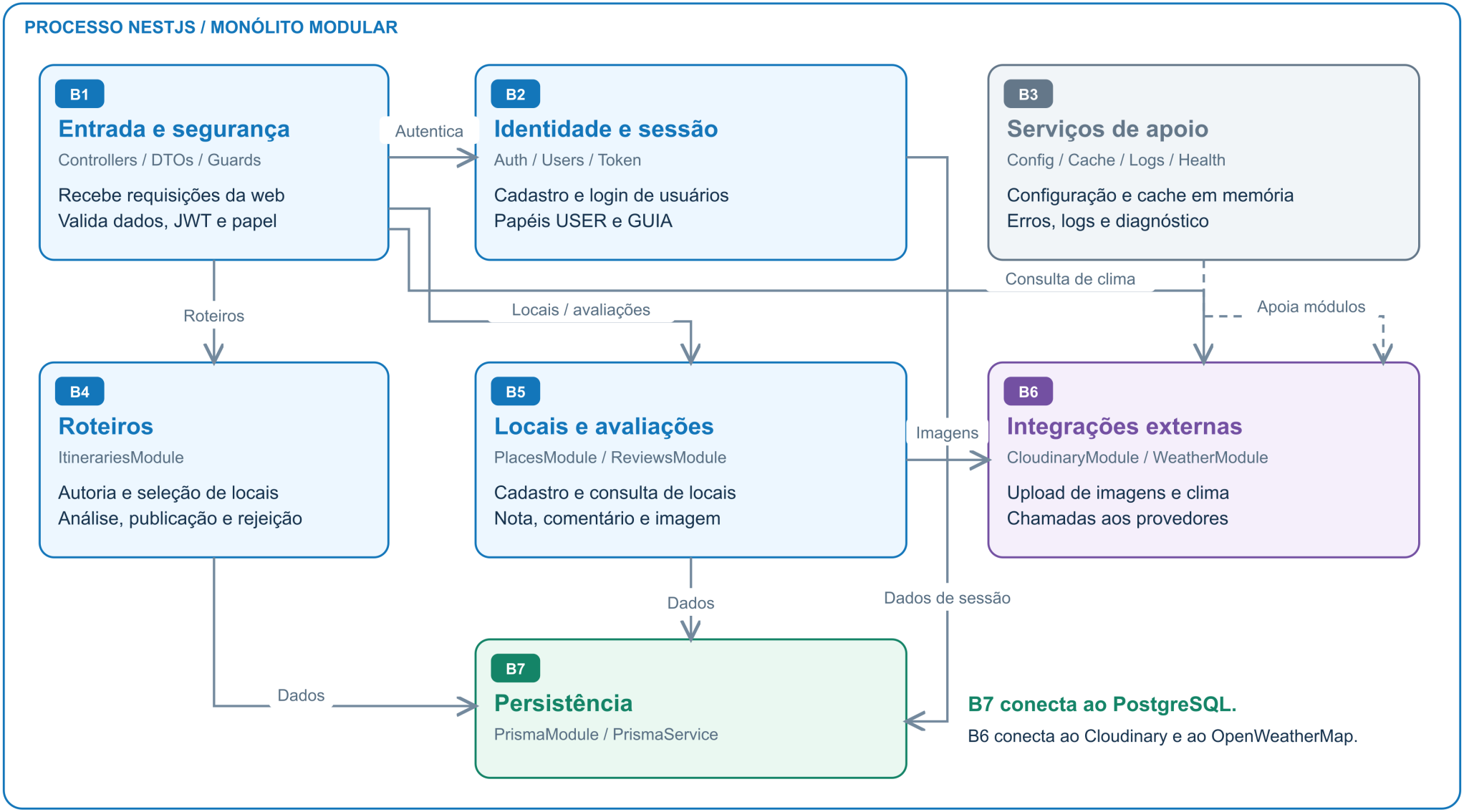
Fornece condições meteorológicas atuais, consultadas pelo backend. A função existe na API, mas a tela de roteiro ainda não a utiliza.

Fluxo principal

A interface envia uma ação à API com o token, quando necessário. A API valida a permissão, aplica as regras e usa o Prisma para persistir os dados. O resultado retorna à interface. No upload, o arquivo vai ao Cloudinary e a URL fica no banco.

Por dentro da API

Visão de componentes | um único backend NestJS, organizado em módulos



Blocos agrupam responsabilidades. Não representam microsserviços ou servidores separados.

Componentes da API

Os módulos dividem responsabilidades dentro de uma única aplicação NestJS.

B1 - Entrada e segurança

Controllers recebem as requisições. DTOs e validações conferem campos e arquivos. Guards verificam o token JWT e, quando necessário, o papel de guia.

B2 - Identidade e sessão

Auth, Users e Token cuidam do cadastro, senha protegida, login, papéis e registro de tokens. O gerenciamento de usuários disponível permite somente listar contas.

B3 - Serviços de apoio

Config carrega variáveis de ambiente; cache em memória reaproveita algumas consultas; logs e filtros registram e padronizam erros; Health expõe o diagnóstico em /health. Swagger documenta a API em /api-docs quando habilitado.

B4 - Roteiros

Itineraries controla autor, título, duração, locais e situação. Um roteiro em análise não pode ser editado. O autor envia para análise e o guia publica, rejeita ou despublica.

B5 - Locais e avaliações

Places mantém os pontos turísticos. Reviews registra nota, comentário, autor, local e imagem opcional. O comentário é parte da avaliação.

B6 - Integrações externas

CloudinaryModule encapsula o upload de imagens. WeatherModule consulta OpenWeatherMap e trata falhas do provedor. Mapas não passam por esse módulo: o navegador consulta OpenStreetMap.

B7 - Persistência

PrismaService é usado pelos módulos que consultam ou alteram dados. O schema Prisma define entidades e relacionamentos no PostgreSQL. Não é um servidor separado.

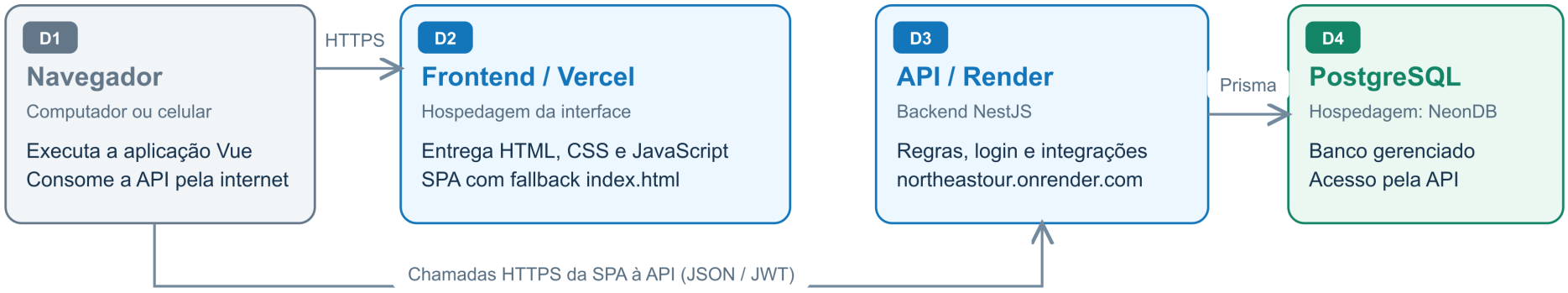
Organização

O monorepo usa npm workspaces. Controllers recebem chamadas; services executam regras; o Prisma concentra a persistência. O cache é local à API. Nenhum bloco interno representa um servidor independente.

Arquitetura de deploy

Ambiente publicado + ambiente local | verificação em 09/11/2026

AMBIENTE PUBLICADO



Frontend: northeas-tour-frontend.vercel.app

Vercel e URL Render identificadas no site público; API respondeu 200 em consulta de locais.

AMBIENTE LOCAL / DOCKER COMPOSE



Cloudinary, OpenStreetMap e OpenWeatherMap continuam externos, conforme a visão geral.

Docker Compose usa alvos dev. Não foram encontrados pipeline CI/CD nem configuração versionada do deploy Render.

Deploy e execução local

Ambientes publicados e locais possuem formas diferentes de execução.

D1 - Navegador

Baixa a interface e executa o Vue. Depois faz chamadas HTTPS diretamente à API, enviando o token nas operações protegidas.

D2 - Vercel

Hospeda o frontend em northeas-tour-frontend.vercel.app. A configuração vercel.json redireciona rotas da SPA para index.html.

D3 - Render

Hospeda a API configurada no frontend em northeastour.onrender.com. O backend atende chamadas, aplica regras e consulta banco e integrações.

D4 - PostgreSQL / NeonDB

Banco PostgreSQL hospedado no NeonDB. A API no Render acessa o banco por DATABASE_URL, utilizando o Prisma.

L1 - Frontend

Container com Vite, disponível por padrão em localhost:5173. VITE_API_URL aponta para localhost:3000 porque a chamada à API parte do navegador no computador do usuário.

L2 - Backend

Container NestJS, publicado em localhost:3000. Acessa o banco pelo nome postgres e porta interna 5432. Na inicialização atual gera o cliente Prisma, sincroniza o schema com db push e inicia o modo de desenvolvimento.

L3 - PostgreSQL

Container postgres:15-alpine. Publica a porta 5433 do computador para 5432 do container; o volume postgres_data mantém os dados.

Deploy observado

Vercel e URL Render identificadas em 09/11/2026 no site público. A API respondeu HTTP 200 em GET /places?limit=1. O PostgreSQL é hospedado no NeonDB, conforme informado pelo responsável pelo projeto. O Docker Compose usa alvos dev; não foi encontrado pipeline CI/CD versionado.

Configuração e fontes

Como os blocos se conectam em cada ambiente.

Endereços publicados

Frontend: https://northeas-tour-frontend.vercel.app
API: https://northeastour.onrender.com
Evidência registrada em docs/diagramas/deploy-verificado.json.

Variáveis da comunicação

VITE_API_URL define o destino das chamadas do frontend no build. CORS_ORIGIN permite a origem do frontend no backend. DATABASE_URL conecta a API ao PostgreSQL no NeonDB em produção e ao serviço postgres no Docker local. JWT_SECRET e variáveis de expiração configuram a autenticação. CLOUDINARY_* e OPENWEATHER_API_KEY ficam no ambiente do backend.

Docker local e produção

Localmente, o navegador usa localhost:5173 para a interface e localhost:3000 para a API. A API usa postgres:5432 dentro da rede Docker. Os Dockerfiles também possuem alvos prod: Nginx para a interface e Node.js para a API; não foi confirmado que o Render os utiliza.

Limites da versão

A tela de roteiro ainda não usa o módulo de clima. Gerenciar usuários permite somente listar contas. A documentação representa o código analisado; o commit implantado na nuvem não foi identificado.

Fontes do projeto

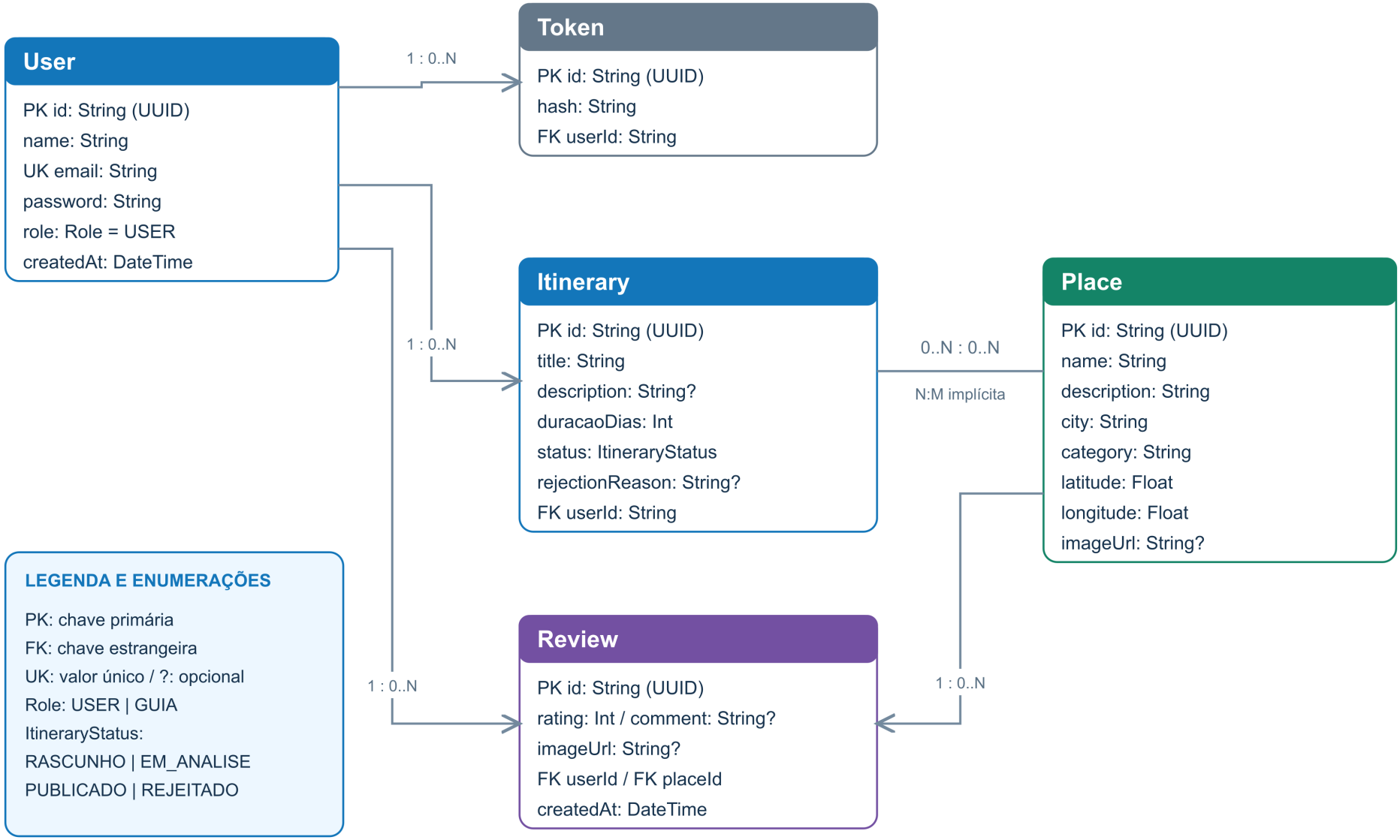
github.com/SeridoTour/SeridoTour
Base local main: 0dd906a31a582a196f68c03534e222574416b1d7.
Arquivos: docker-compose.yml; Dockerfiles; vercel.json; AppModule; main.ts; cliente Axios; schema.prisma.

Referência visual

Modelo C4: c4model.com/diagrams/container e c4model.com/diagrams/deployment. Os desenhos são próprios e usam notação livre de blocos, permitida pelo enunciado.

Estrutura de dados

DER lógico atualizado | nomes e campos do schema Prisma



A junção entre Itinerary e Place é gerenciada pelo Prisma; não foi criada uma entidade associativa fictícia.

Entidades e relacionamentos

DER atualizado a partir de apps/backend/prisma/schema.prisma.

User - usuário

Armazena nome, e-mail único, senha protegida, papel e data de cadastro. Seu id identifica o autor dos roteiros e das avaliações. Pode possuir zero ou vários tokens, roteiros e avaliações.

Token - registro de token

Contém id, hash e userId. Cada token pertence a exatamente um usuário. A estrutura permite mais de um registro por usuário; o serviço atual substitui o token utilizado no login.

Place - local turístico

Contém nome, descrição, cidade, categoria, coordenadas e URL opcional de imagem. Pode receber várias avaliações e participar de vários roteiros.

Itinerary - roteiro

Contém título, descrição opcional, duração em dias, situação, motivo opcional de rejeição e autor. O estado inicial é RASCUNHO. Outros estados são EM_ANALISE, PUBLICADO e REJEITADO.

Review - avaliação

Contém nota, comentário, imagem opcional, autor, local e data. Cada avaliação pertence a um usuário e a um local. O formulário exige comentário e nota de 1 a 5; o banco permite comentário nulo.

Relação entre roteiros e locais

Itinerary e Place têm relação muitos-para-muitos. Um roteiro pode usar vários locais e um local pode participar de vários roteiros. O Prisma administra a junção implicitamente; não há modelo associativo com id próprio declarado no schema.

Estrutura atual

O modelo inclui status e rejectionReason do roteiro e imageUrl de local e avaliação. A relação entre roteiros e locais é administrada implicitamente pelo Prisma, sem uma entidade intermediária com identificador próprio no schema.